Numerical Runtime Intelligence (NRI): A Framework for Real-Time Observation, Classification, and Selective Stabilization of Numerical Instability Propagation in GPU Transformer Runtimes

A Unified Framework via Shannon Entropy of Already-Computed Distributions

Makhebi Ahlem
Independent Researcher
Germany
ahlemmakhebi@protonmail.com
ORCID: 0009-0007-7010-3282

Version 1.0.0 — May 2026
Project: **nonansNRI** Reference implementation: `nonans` v0.4.0
Repository: https://github.com/makheahlem/nonans Benchmark seed: `20260525`

This document establishes the conceptual architecture, formal framework, benchmark results, and attribution record for the Numerical Runtime Intelligence (NRI) framework and its reference implementation ***nonans*** *v0.4.0. The terminology, named architectural components, and benchmark protocols defined herein constitute the founding technical vocabulary of this work.*

Abstract

We introduce Numerical Runtime Intelligence (NRI), a runtime instrumentation layer for the real-time classification of legitimate numerical events—non-finite values that arise from the computation itself (overflow, structural rank collapse, attention concentration), not from hardware faults. NRI is distinct from algorithm-based fault tolerance, which targets bit-level corruption; from static floating-point precision analysis, which operates offline on operator code; and from training-stability interventions, which prevent rather than classify. The contribution is a typed structural interface to the runtime, not a detector, predictor, or optimizer.

The framework rests on a single mathematical identity: Shannon entropy $H(p) = -\sum_i p_i \log p_i$, applied to distributions *already computed* during normal operation, serves as a structural-state signal for weight tensors at training time and as an attention-state signal at inference time. The two are the same normalized functional applied to different distributions naturally arising at each stage; we verify the numerical equivalence to 3.18×10^{-7} max pointwise error over $N = 10\,000$ random instances.

The open instrumentation layer (`nonans` v0.4.0) implements a three-gate cascade architecture: a norm proxy at $O(n)$ cost every step, a randomized-SVD structural scan at $O(mnk)$ cost on trigger or periodically, and an $O(1)$ context query at the singularity boundary. Measured CPU overhead of entropy computation on already-resident softmax tensors is 15.39% mean (range 9.8–22.7%) across five model configurations, on an Intel Xeon @ 2.80 GHz; GPU overhead is not measured in this release, and the measurement protocol is provided.

On a controlled-instability protocol with fixed seed `20260525`, the NRI signal score produces a mean structural precursor lead of 68.8 steps before a `NaN` event ($n = 30$, 30/30 detection, 0% false-alarm rate). Comparative evaluation against gradient-norm, loss-spike, and weight-norm baselines shows that the signal score achieves 84.5 steps precursor lead vs. 54.9 for gradient-norm monitoring at equal false-alarm rate—a 29.6-step advantage arising from the window between structural collapse and its magnitude manifestation.

The framework distinguishes transient ghost faults (which self-correct) from persistent real faults (which do not), via a three-signal majority classifier grounded in the dynamics of restoring-force depletion. At inference time, a short calibration pass enables the finer DEVIATION class while maintaining 100% detection of collapse and maximum-entropy attention, at a measurable in-distribution flag cost that we report rather than suppress.

All results are bounded, reproducible, and accompanied by exact measurement protocols. The reference implementation is released as an Apache 2.0-licensed PyTorch library with a complete NumPy algebraic validation suite, six benchmarks, and four reproducibility protocols. A proprietary resolution layer, maintained in a separate private codebase, consumes the typed structural interface at the singularity boundary and returns a defined value so that computation continues in place. Its method and empirical evaluation are deliberately outside the scope of this paper and are the subject of separate forthcoming work; the contribution of the present work is the open instrumentation and classification layers evaluated above.

Contents

1 Introduction **4**
- 1.1 The infrastructure problem . . . 4
- 1.2 The NRI layer . . . 4
- 1.3 Contributions . . . 4
- 1.4 NRI as an intelligence interface . . . 5
- 1.5 Scope and honesty . . . 5

2 Background and Related Work **5**

3 Unified Entropy Framework **7**
- 3.1 Formal setting . . . 7
- 3.2 Unified identity . . . 7
- 3.3 Instability propagation causal chain . . . 8
- 3.4 Health state classification . . . 8
  - 3.4.1 Training-time states . . . 8
  - 3.4.2 Inference-time states . . . 8

4 System Design **9**
- 4.1 Three-gate architecture . . . 9
- 4.2 FaultContext interface . . . 9
- 4.3 Telemetry bus . . . 10
- 4.4 RuntimeMonitor — inference-time . . . 10
- 4.5 Overhead model . . . 10

5 Benchmark Results **11**
- 5.1 B1: Unified identity verification . . . 11
- 5.2 B2: Fault class detection . . . 11
- 5.3 B3: CPU entropy overhead . . . 11
- 5.4 B4: Early-warning lead time . . . 12
- 5.5 B5: Comparative detector evaluation . . . 12
- 5.6 B6: Calibration effect . . . 13

6 Outstanding Measurements **14**

7 Failure Taxonomy **14**

8 Resolution Boundary **15**

9 Open Source Strategy and Release Lineage **15**
9.1 Open/proprietary boundary 15
9.2 Version lineage 15
9.3 Repository structure 15

10 Licensing Philosophy and Commercial Pathways **15**

11 Provenance and Attribution **16**
11.1 Technical vocabulary as attribution anchor 16
11.2 Version-anchored benchmark results 17
11.3 Fabrication note 17

12 Conclusion **17**

A Reproducibility Protocols **17**

B Library API Reference **17**

C Recommended Citation **18**

1 Introduction

1.1 The infrastructure problem

Training large transformer models at scale is punctuated by catastrophic numerical events: `NaN`/`Inf` propagation through weight tensors, silent rank collapse that degrades representational capacity without triggering an exception, and attention fixation at inference time that corrupts generation quality without surface-level evidence.

The standard production toolkit is reactive and magnitude-based. Gradient-norm clipping fires after structural degradation has already propagated to a scalar norm [Goodfellow et al., 2016]. AMP loss scaling reacts to the loss surface after weight corruption is underway [Micikevicius et al., 2018]. Both mechanisms fire on symptoms, not on structural causes.

The fundamental limitation is architectural: magnitude-based monitors observe scalar projections of the tensor state. They discard the *structural* information in the distribution of singular values—information that changes measurably earlier in the causal chain than any scalar norm.

1.2 The NRI layer

Numerical Runtime Intelligence occupies a defined position in the GPU systems stack:

`Training framework`	`PyTorch, FSDP, DeepSpeed, JAX`
`Resolver (Layer C)`	`Proprietary stabilization layer`
`NRI --- this work`	`Observability + Classification`
`GPU hardware`	`Receives finite, coherent tensors`

The NRI layer is the contract between what the training framework produces and what the stabilization layer can act on. Without NRI, the stabilization layer operates blindly: it knows a `NaN` occurred but not why, where the structural fault originated, or whether the fault would have self-corrected. With NRI, stabilization becomes context-informed.

1.3 Contributions

The mathematical primitives this work builds on are not new: Shannon entropy, the SVD-entropy formulation of structural complexity, and attention entropy as an instability indicator have all appeared previously (see Section 2). The contributions of this work are therefore framed as a systems-level integration of these primitives into a runtime instrumentation layer, not as a discovery of the primitives themselves. Specifically:

1. **Unified-identity interface.** We show that the training-time singular-value entropy and the inference-time attention entropy are the same normalized Shannon functional applied to different distributions naturally arising at each stage, and we verify this numerical equivalence to 3.18×10^{-7} max pointwise error over $N = 10\,000$ samples. Prior work tracks each quantity in isolation; the contribution is the unified contract that lets a single classifier and a single threshold semantics operate across both regimes.

2. **Runtime propagation classification.** The primary intellectual contribution is the classification of an evolving structural trajectory into propagation regimes—specifically the ghost/real distinction derived from restoring-force depletion (collapse rate, gradient magnitude, momentum alignment) rather than from a threshold crossing on a scalar. Prior work on attention-entropy collapse and on spectral evolution of weights observes the relevant geometric phenomena; this work converts the observed trajectory into a discrete runtime state usable by a fault-handling boundary.

3. **Zero-additional-cost integration as a runtime layer.** We describe a three-gate cascade that bounds instrumentation overhead by computing the structural signals from quantities already resident in the forward pass (softmax outputs, top-k singular values from a periodic randomized SVD), produces a structured `FaultContext` record at fault time, and exposes the classification result through a stable interface. Prior real-time training-dynamics tools (Section 2) target diagnostics over hidden representations; the contribution here is the framing as an operational telemetry layer that adds no computation beyond what monitoring already performs.

4. **Reproducible benchmark suite (B1–B6).** Six deterministic benchmarks with fixed seed (`20260525`) and four PyTorch measurement protocols (P1–P4) for community extension. The benchmark results in this paper are the empirical basis for the claims above; what is not measured here (GPU overhead, large-model validation, distributed settings) is explicitly listed in Section 6.

We are explicit about what is *not* claimed as novel: the use of Shannon entropy as an instability indicator (a long-standing technique in dynamical-systems monitoring), attention-entropy collapse as a training-stability symptom (established by Zhai et al. 2023), the spectral evolution of weight matrices during training (Martin and Mahoney 2021, Yunis et al. 2024), and real-time visualization of training dynamics (Yang and Dong 2024). The contribution is the integration of these into a unified runtime interface with classified output.

1.4 NRI as an intelligence interface

The layers described above are best understood not as a monitor but as an *intelligence interface* to the numerical runtime: a contract that renders the internal structural state of a running computation legible and delivers it, classified and with provenance, to whatever consumes it. The observability layer extracts structural signals; the classification layer interprets their trajectories as propagation regimes; the resulting `FaultContext` is a structured statement about *what is happening to the numerical state and why*, not a raw measurement. This interface is the published contribution of the present work. What acts on it—the resolution layer at the runtime boundary—consumes the interface without being part of it, and is treated separately (Section 8).

1.5 Scope and honesty

We are deliberate about what this paper does and does not claim. The benchmark results are real and bounded: the controlled-instability protocol is controlled, not a live large-model training run; the overhead measurements are CPU-only; the ghost/real classifier thresholds are analytically derived, not fitted to a model zoo. Missing measurements are documented as missing, with protocols provided. The proprietary resolution layer is sealed: declared at the runtime boundary (Section 8) but with its method and evaluation deferred to separate forthcoming work.

2 Background and Related Work

Shannon entropy. Shannon entropy $H(p) = -\sum_i p_i \log p_i$ measures concentration of a probability distribution [Shannon, 1948]. The normalized form $\tilde{H}(p) = H(p)/\log k \in [0, 1]$ is bounded on the unit interval. Two properties of entropy are directly relevant here. First, entropy responds to concentration, not magnitude: a tensor that doubles in norm but maintains uniform singular-value spread has the same structural health signal before and after scaling; a tensor that loses rank while maintaining norm has a falling signal. Second, entropy is computable from distributions already produced in the training and inference forward passes at $O(k)$ or $O(n)$ additional cost on data already in cache.

Attention and rank collapse. Vaswani et al. [2017] introduced scaled dot-product attention with a softmax output defining a probability distribution over n key positions. Dong et al. [2021] proved that stacking attention layers without skip connections collapses input rank doubly exponentially with depth due to the doubly stochastic structure of the attention operator. The rank-collapse trajectory identified by Dong et al. [2021] is one of the controlled perturbation profiles in our B4/B5 protocol.

Floating-point precision repair in DL operators. Liu et al. [2025] address floating-point precision problems in CNN operators through automated detection and repair, targeting accumulated floating-point error in linear operator chains. Their work shares the broad concern of legitimate numerical events in deep-learning kernels with the present work, but operates as a static-analysis-and-repair tool over specific operators rather than as a runtime classification interface; the contribution here is orthogonal in surface (runtime classification vs. analysis-and-repair).

Training stability monitoring. Gradient-norm monitoring [Goodfellow et al., 2016] fires when the gradient magnitude exceeds a threshold. AMP loss scaling [Micikevicius et al., 2018] reacts to non-finite loss. Kloberdanz et al. [2022] characterize numerical-stability bugs in PyTorch and TensorFlow and the common patterns of unstable algorithm implementations (e.g., overflow in softmax), motivating runtime instrumentation but not proposing one. NRI introduces structural monitoring via spectral entropy on already- resident distributions, orthogonal to magnitude-based monitors.

Attention entropy and spectral dynamics during training. The closest work is Zhai et al. [2023] (σReparam), which tracks attention entropy and the largest singular value of attention weights during transformer training and links sharp drops in attention entropy to training instability. σReparam is an *intervention* (a reparameterization that suppresses entropy collapse) rather than an observability layer; the quantities it tracks are tracked for diagnostic and regularization purposes. Martin and Mahoney [2021] and Yunis et al. [2024] characterize the singular-value spectrum evolution of weight matrices over training, identifying phase transitions and effective-rank decline across architectures. These works establish the geometric phenomena that the present paper observes in real time and classifies into discrete propagation regimes; they do not propose a runtime classification layer or a fault-handling interface.

SVD-entropy as an instability indicator in other domains. The use of Shannon entropy of a singular-value distribution as an instability indicator predates its use in deep learning. Rodriguez et al. [2022] apply this construction to nuclear-reactor power oscillations in boiling water reactors and find that instabilities correspond to marked reductions in SVD entropy (equivalently, marked increases in distance to randomness). Zhang et al. [2022] use wavelet singular spectral entropy for compressor-stall precursor detection in axial-flow compressors. The mathematical core is therefore established prior art outside ML; the present work transfers the construction to transformer runtimes and integrates it with a runtime classification layer.

Randomized SVD. Halko et al. [2011] introduced the probabilistic range-finder (Algorithm 4.1) enabling truncated SVD at $O(mnk)$ instead of $O(\min(m,n)^2 \max(m,n))$. The cost reduction makes on-the-fly SVD feasible in the training loop. Our Gate 2 uses this algorithm with $k = 8$ singular modes by default.

Real-time training-dynamics visualization. Yang and Dong [2024] (the SentryCam framework) present a live-update visualization framework for the evolution of hidden representations during training, motivated by the observation that conventional metrics are lagging indicators of

model health. SentryCam targets diagnostic visualization over hidden representations; the contribution here is complementary and distinct in surface: a low-overhead operational telemetry layer over the *already-computed* structural distributions, exposed through a classified `FaultContext` interface rather than a visualization dashboard.

Entropy as an inference signal. Manakul et al. [2023] uses output-distribution entropy to detect hallucination in generation; this is orthogonal to the present work—they measure entropy of the generated token distribution, we measure entropy of structural distributions internal to the model. Spectral normalization [Miyato et al., 2018] and implicit regularization analysis [Arora et al., 2019] use SVD in training but not as a real-time health signal with explicit fault classification or ghost/real discrimination.

3 Unified Entropy Framework

3.1 Formal setting

A neural network training run defines a discrete-time dynamical system. At step t, the system state is $\mathcal{S}(t) = \{W_i(t)\}_{i=1}^{N}$ where $W_i \in \mathbb{R}^{m_i \times n_i}$ are the parameter tensors, evolved by:

$$\mathcal{S}(t+1) = \mathcal{F}(\mathcal{S}(t),\, \nabla\mathcal{L}(t),\, \text{state}_{\text{opt}}(t)).$$

Standard training infrastructure treats this evolution as a black box, observing only scalar projections: $\mathcal{L}(t)$, $\|\nabla\mathcal{L}\|_2$, and occasionally $\|W_i\|_F$. These are magnitude projections; they discard structural information.

Definition (structural state). The *structural state* of parameter W_i at step t is the normalized singular-value distribution:

$$p_i(t) = \frac{\sigma_i(t)}{\|\sigma_i(t)\|_1}, \quad \sigma_i(t) = \text{top-}k \text{ singular values of } W_i(t).$$

$p_i(t)$ is a probability distribution over k singular modes. It carries rank-structural information that $\|W_i\|_F$ cannot: whether representational capacity is spread across modes (healthy) or collapsed toward a low-rank projection (structurally degenerate).

3.2 Unified identity

Proposition 1 (Unified Identity)**.** *Let the training-time* signal score *be:*

$$s(\sigma) = \frac{-\sum_i p_i \log p_i}{\log k}, \quad p_i = \frac{\sigma_i}{\|\sigma\|_1},$$

and the inference-time normalized attention entropy *be:*

$$\tilde{H}(q) = \frac{-\sum_i q_i \log q_i}{\log n}, \quad q_i = \frac{a_i}{\|a\|_1}.$$

When applied to the same input distribution $v/\|v\|_1$, $s(v) \equiv \tilde{H}(v/\|v\|_1)$ up to floating-point precision.

Proof. Both are $H(p)/\log k$ where $H(p) = -\sum_i p_i \log p_i$ and $p = v/\|v\|_1$. The functional is identical; the distributions differ at training vs. inference time. □

Remark 1. *Proposition 1 is a definitional identity, not a deep theorem: once both quantities are defined as the normalized Shannon entropy of a normalized non-negative vector, their equality on a shared input is immediate. The content of the unified-identity claim is not mathematical*

but architectural — that the same *normalized functional, computed on distributions that arise naturally at two different stages (the singular-value distribution of a weight tensor at training time, the attention distribution at inference time), can serve as a single structural-health signal under one threshold semantics and one classifier. The numerical verification below confirms the two independent implementations agree to floating-point precision; it does not claim to prove anything beyond implementation consistency.*

Numerical verification (Benchmark B1). Over $N = 10\,000$ random distributions sampled from a symmetric Dirichlet distribution with concentration parameter $\alpha \in \{0.1, 0.5, 1.0, 2.0, 10.0\}$, the maximum absolute pointwise error between independently implemented $s(\cdot)$ and $\hat{H}(\cdot)$ is 3.18×10^{-7}; mean error 2.31×10^{-8}. All errors are attributable to floating-point arithmetic, not algorithmic divergence. Fixed seed: `20260525`.

3.3 Instability propagation causal chain

Catastrophic numerical events follow a four-stage causal sequence:

Stage 0 — Structural deformation onset. The singular-value distribution of one or more parameters begins to concentrate: $s(t)$ declines. The gradient norm is flat; the loss surface has not yet reflected the structural change. This stage is only visible to NRI.

Stage 1 — Restoring force depletion. The optimizer's first moment (m_t in Adam) begins to lock onto the collapse direction. The cosine similarity $\langle m_t, g_t \rangle$ becomes negative: the first moment is now actively reinforcing collapse. The gradient magnitude $\|g_t\|_2$ falls below the threshold that would restore spectral spread. The structural deformation is no longer self-correcting. This is the *ghost-to-real transition.*

Stage 2 — Magnitude manifestation. Structural collapse amplifies into a measurable scalar change. Gradient-norm and weight-norm monitors first become aware of the fault at this stage. The 29.6-step precursor advantage (Benchmark B5) is consistent with the gap between Stage 1 and Stage 2; we do not measure the two stages independently, so this correspondence is an interpretation of the lead-time result, not a separately verified quantity.

Stage 3 — Catastrophic event. The accumulated instability manifests as a `NaN/Inf` event. Resolution, if available, must act on whatever context has been accumulated before this point.

3.4 Health state classification

3.4.1 Training-time states

Table 1: Training-time health state classification.

State	Condition	Interpretation
UNKNOWN	History < 5 scans	Insufficient trajectory
HEALTHY	$s \geq 0.72$	Spectral mass distributed
GHOST	$s \in [0.15, 0.72)$, < 2 signals	Self-correcting deformation
REAL	$s < 0.15$, or ≥ 2 signals	Restoring force exhausted

The three fault signals in the ghost/real classifier are: (i) collapse rate $\dot{s} < -0.025$, (ii) restoring force $\|g\|_2 < 10^{-4}$, (iii) momentum alignment $\cos(m_t, g_t) < -0.3$. REAL requires ≥ 2 of three, except at the extremes: $s \geq 0.72$ always yields HEALTHY; $s < 0.15$ always yields REAL.

3.4.2 Inference-time states

COLLAPSE and MAXIMUM are detectable without calibration. DEVIATION detection requires a short calibration pass (≥ 500 samples; B6).

Table 2: Inference-time `RuntimeHealth` classification.

State	Condition	Interpretation
UNKNOWN	No calibration, partial state	Indeterminate
HEALTHY	$H \in [\text{cal}_5, \text{cal}_{95}]$	Nominal distribution
COLLAPSE	$H < \theta_{\text{collapse}}$	Single-position fixation
MAXIMUM	$H > \theta_{\text{maximum}}$	Near-uniform attention
DEVIATION	$\lvert H - \bar{H} \rvert > 3\sigma_{\text{cal}}$	Outside calibrated range

4 System Design

4.1 Three-gate architecture

Algorithm 1 NRI Three-Gate Monitoring Engine (`nonans` v0.4.0)

for each training step t **do**
  for each tracked parameter W_i **do**
    $\delta_i \leftarrow |\|W_i\|_F - \|\bar{W}_i\|_{\text{base}}| / \|\bar{W}_i\|_{\text{base}}$
    if $\delta_i > \tau_1$ **or** $\|W_i\|_F \notin \mathbb{R}$ **or** $t \bmod$ `scan_every` $= 0$ **then**
      Gate 2: $\sigma \leftarrow \text{RandomizedSVD}(W_i, k)$
      Compute $s, \dot{s}, \|g\|_2, \cos(m_t, g_t)$
      $h \leftarrow$ `classify`$(s, \dot{s}, \|g\|_2, \cos(m_t, g_t))$
      Write `FaultContext`$(W_i, t, h, \sigma, \sigma_{\text{healthy}}) \rightarrow$ `Registry`
      Emit `GATE2_COMPLETE` event to `TelemetryBus`
    end if
  end for
end for
On `NaN/Inf` **at** W_j**:**
  Gate 3: `ctx` $\leftarrow$ `Registry.get`(W_j) ▷ $O(1)$, RLock-protected
  Dispatch `ctx` to `Resolver` within `budget_ms` (default 5ms)

Gate 1 ($O(n)$, every step, all tracked parameters). Norm deviation proxy: fires when $\delta_i > \tau_1 = 0.12$ (configurable) or when the norm is non-finite. Does not classify; only gates Gate 2 to bound SVD cost. Also fires unconditionally on non-finite norms regardless of τ_1.

Gate 2 ($O(mnk)$, on Gate 1 fire or every `scan_every` $= 25$ steps). Randomized SVD [Halko et al., 2011] with $k = 8$. Computes five structural signals: signal score s, shape score s_{shape}, collapse rate $\dot{s}$, restoring force $\|g\|_2$, and momentum alignment $\cos(m_t, g_t)$. Passes to the three-signal ghost/real classifier. Writes a complete `FaultContext` record to the RLock-protected context store. Emits events to the `TelemetryBus`.

Gate 3 ($O(1)$, at singularity only). RLock-protected dictionary lookup returning the most recent `FaultContext` for the faulting tensor. The resolver receives structural provenance at the moment of intervention rather than a bare scalar fault signal. Returns `None` if no scan has been completed for that tensor; the resolver falls back to its default uninformed strategy.

4.2 FaultContext interface

The `FaultContext` dataclass is the primary interface between the open monitoring layer (Layers A and B) and the proprietary resolver layer (Layer C):

- Tensor identity (module path, parameter name), shape, step index.
- Health classification (HEALTHY/GHOST/REAL) and confidence at the time of the scan.

- Five structural signals: signal score s, shape score s_{shape}, collapse rate $\dot{s}$, restoring force $\|g\|_2$, momentum alignment $\cos(m_t, g_t)$.
- Current top-k singular values (float32 vector, CPU-side).
- Last-healthy singular values: the most recent snapshot where $s_{\text{shape}} \geq 0.72$. Provides a structural recovery target for the resolver.
- Shape-score trajectory over the `window` (float32 tensor, 12 steps by default).
- Gate 2 compute time (for overhead tracking).

The `adjusted_confidence(current_step)` method linearly decays confidence with step age, reaching zero at `max_age` = 50 steps, preventing stale contexts from driving over-confident resolutions.

The `FaultContext` is the open-layer contract. The proprietary resolver implements `ResolverProtocol` (the abstract base class) using this context without modifying any open-layer code. The two layers are cleanly separated.

4.3 Telemetry bus

All Gate 2 completions, health-state transitions, singularity detections, and resolution events are emitted to a `TelemetryBus`: a publish-subscribe multiplexer with pluggable sinks (`InMemorySink`, `CallbackSink`, or custom `TelemetrySink` implementations). The bus is process-global by default; additional instances support test isolation. Telemetry failure never propagates to the training loop.

4.4 RuntimeMonitor — inference-time

The `RuntimeMonitor` applies the same entropy functional to attention distributions at inference time.

Uncalibrated mode: detects absolute COLLAPSE ($H < 0.2$) and absolute MAXIMUM ($H > 0.95 \ln n$) from absolute thresholds, without calibration data. 100% OOD detection; in-distribution attention correctly classified HEALTHY (Benchmark B6).

Calibrated mode: after ≥ 500 in-distribution samples, learns per-head nominal distributions (mean, std, P5, P95). Enables the finer DEVIATION class while maintaining 100% OOD detection; the added 3σ sensitivity flags a minority ($\approx 12.5\%$ on the B6 synthetic set) of in-distribution samples as DEVIATION, a trade-off tuned via the σ multiplier (Benchmark B6).

The hot path uses `attention_entropy_batched`: a vectorized entropy computation over the full head dimension in a single tensor operation, avoiding per-head Python overhead in the inference forward pass.

4.5 Overhead model

Per tracked parameter:

$$\begin{aligned}\text{Gate 1 cost} &= O(n) \text{ per step, all parameters} \\ \text{Gate 2 cost} &= O(mnk) \text{ per trigger or period} \\ \text{Gate 3 cost} &= O(1) \text{ at fault time}\end{aligned}$$

The Gate 2 cost is dominated by the $A\Omega$ sketch product in the randomized range-finder (Halko et al. 2011, Algorithm 4.1): for an $m \times n$ matrix sketched to k modes this is $O(mnk)$. For a 768×768 projection matrix with $k = 8$, the sketch product is approximately $768 \cdot 768 \cdot 8 \approx 4.7 \times 10^6$ multiply-adds, plus a thin QR and a small $k \times n$ SVD whose costs are lower order. Amortized over the default `scan_every` = 25 steps, the per-step Gate 2 cost is $O(N_{\text{param}} \cdot mnk/25)$.

Memory per tracked parameter: ≈ 224 bytes (norm ring-buffer 144 bytes; shape ring-buffer 48 bytes; last-healthy singular values 32 bytes). For 50,000 tracked parameters: ≈ 11 MB. This is negligible relative to model weight memory at any realistic scale.

5 Benchmark Results

All benchmarks use fixed seed `20260525`, NumPy `default_rng` with this seed for all random draws, deterministic perturbation protocols with documented parameters, and produce identical output on all IEEE-754-compliant hardware. Run order: B1 → B2 → B3 → B4 → B5 → B6. Benchmarks are independent (no shared state). JSON output alongside PNG figures will be added in v0.5.0.

Table 3: Benchmark suite summary (`nonans` v0.4.0, seed `20260525`).

Benchmark	Claim tested	Result	§
B1: Unified identity	$s(\cdot) \equiv \tilde{H}(\cdot)$	3.18×10^{-7} max error, N=10 000	5.1
B2: Fault detection	100% on 3 canonical fault classes	100%, N=1000 per class	5.2
B3: CPU overhead	Entropy / attention forward pass (CPU)	15.39% mean (9.8–22.7%)	5.3
B4: Lead time	Precursor gap, controlled protocol	68.8 steps mean, 30/30, 0% FA	5.4
B5: Detector comparison	Signal score vs. baselines	84.5 vs. 54.9 steps at 0% FA	5.5
B6: Calibration	FA rate before/after calibration	1.00 → 0.00 after 500 samples	5.6

5.1 B1: Unified identity verification

$N = 10\,000$ random distributions sampled from a symmetric Dirichlet with $\alpha \in \{0.1, 0.5, 1.0, 2.0, 10.0\}$ (2,000 instances per level). Signal score $s(\cdot)$ and normalized attention entropy $\tilde{H}(\cdot)$ are implemented independently in the NumPy algebraic mirror (`nonans_numpy_mirror.py`).

Maximum absolute pointwise error: 3.18×10^{-7}. Mean absolute pointwise error: 2.31×10^{-8}. All errors attributable to floating-point arithmetic only. The functional identity (Proposition 1) is confirmed at numerical precision.

5.2 B2: Fault class detection

Three canonical fault classes: rank-collapse (progressive drain of spectral mass into the dominant mode), attention-fixation (near-one-hot attention softmax), gradient-explosion (multiplicative scale amplification). $N = 1000$ instances per class across five model configurations.

Detection rate: 100% across all three classes and all configurations. Zero misclassifications; zero missed detections. The entropy threshold provides clean separation between fault-class distributions and the in-distribution baseline.

5.3 B3: CPU entropy overhead

Entropy computation overhead measured relative to the attention forward pass on CPU (NumPy algebraic mirror). Five model configurations: $\{8, 8, 16, 16, 32\}$ heads $\times$ $\{128, 256, 256, 512, 512\}$ sequence length. Each configuration: 500 repetitions; 10 warmup passes excluded.

The overhead ratio declines monotonically with model size, consistent with entropy being $O(n)$ while attention is $O(n^2)$. GPU overhead is not measured in this release. Protocol P2

Table 4: CPU entropy overhead relative to attention forward pass (B3). Measured on Intel Xeon @ 2.80 GHz, Python 3.12, NumPy 2.4, seed `20260525`. Timing results are hardware-dependent; these figures are illustrative of the CPU overhead profile, not a canonical constant.

Configuration	Attn (μs)	Entropy (μs)	Overhead (%)
8 heads × 128	37.4	8.5	22.7
8 heads × 256	53.6	10.7	19.9
16 heads × 256	110.1	15.8	14.3
16 heads × 512	194.5	19.8	10.2
32 heads × 512	360.0	35.3	9.8
Mean			**15.39**

(`run_overhead_torch.py`) enables community measurement on accelerator hardware. The claim of 3–8× lower GPU overhead is an engineering expectation based on bandwidth advantage of on-device computation; it is not a measured result and is not cited as such.

5.4 B4: Early-warning lead time

Controlled-instability protocol. A 32×32 weight matrix with known singular-value structure ($\sigma_i \approx 1.0 - 0.3i/d$). Instability injected by two mechanisms in sequence: (i) *rank drain*: a low-rank outer product aligned to a fixed direction, added to the gradient at each step, progressively draining spectral mass into the dominant singular mode (a synthetic analogue of the rank-collapse phenomenon characterized by Dong et al. 2021, not their procedure); (ii) *momentum amplification*: a multiplicative scale factor growing toward infinity at a controlled rate, simulating Adam momentum locked onto a noise direction. Three profiles (mild, moderate, severe) × 10 seeds = $n = 30$ runs.

Table 5: Early-warning lead time by instability profile (B4). All 30 runs diverge; all 30 trigger an alarm before the NaN event.

Profile	n	Mean (steps)	Median	Range
Mild	10	73.9	74	$[73, 75]$
Moderate	10	73.5	74	$[59, 75]$
Severe	10	59.0	59	$[59, 61]$
All	**30**	**68.8**	**73**	$[59, 75]$

Detection rate: 30/30 = 100%. False-alarm rate on 10 benign random-walk runs: 0%. $[P_{25}, P_{75}] = [59, 74]$. The shorter lead for the severe profile is consistent with the theoretical expectation: faster rank drain leaves less time between first structural signal and magnitude explosion.

Remark 2. *This protocol is controlled, not a real training run. Real lead times depend on model architecture, learning-rate schedule, and hardware precision. Protocols P3 and P4 enable measurement on real training runs and real models. Results from these protocols are not yet available and are not reported here.*

5.5 B5: Comparative detector evaluation

$n = 20$ unstable runs and $n = 20$ benign random-walk runs from the same controlled-instability protocol used in B4. Three baseline detectors evaluated at their canonical thresholds: gradient-

norm spike (5× running median), loss spike (3× running median), weight-norm deviation (> 50%).

Table 6: Detector comparison on controlled-instability protocol (B5). False-alarm rate on 20 benign random-walk runs.

Detector	Det. rate	Lead (steps)	FA rate
Gradient-norm spike (5×med)	1.00	54.9	0.00
Loss spike (3×med)	0.00	—	0.00
Weight-norm deviation (> 50%)	1.00	100.0	0.00
Signal score < 0.40 (ours)	**1.00**	**84.5**	**0.00**
Attn entropy uncalibrated (ours)	1.00	138.9	1.00

The 29.6-step advantage of signal score over gradient-norm monitoring is consistent with the window between Stage 1 (restoring force depletion) and Stage 2 (magnitude manifestation) of the instability propagation sequence (Section 3.3).

One row in Table 6 deserves direct comment: the weight-norm deviation detector posts a longer lead (100.0 steps) than signal score (84.5) on this protocol. We do not claim signal score dominates on raw lead time. The weight-norm detector's long lead here is an artifact of the controlled protocol — the injected multiplicative scale factor inflates the weight norm early and monotonically, which is exactly what a > 50% deviation trigger keys on. On real training runs, where weight norm drifts for many benign reasons, that same sensitivity is what produces the high false-alarm rates that magnitude monitors are known for; the controlled protocol does not exercise that failure mode. The defensible claim is therefore not "earliest detection" but *structural* detection with an interpretable `FaultContext`: the resolver receives the singular-value trajectory, the last-healthy snapshot, trajectory confidence, and the ghost/real classification, converting blind recovery to context-informed recovery. No magnitude detector emits that record.

5.6 B6: Calibration effect

In-distribution calibration uses a moderately concentrated softmax sample (temperature 1.5 in the equivalent parameterization). OOD collapse: near-one-hot (0.98 mass on one position). OOD maximum: near-uniform (softmax temperature 0.05). The classifier applies absolute thresholds in both modes (`collapse_thr_abs` = 0.2, `max_thr_abs` = $0.95 \log n$); calibration adds a per-head nominal band (mean, std, p_5, p_{95}) and a 3σ DEVIATION test.

After 500 calibration samples, the monitor fits a nominal distribution $H \sim \mathcal{N}(3.16, 0.35)$, $[P_5, P_{95}] = [2.56, 3.57]$.

Table 7: Detection accuracy with and without calibration (B6), $N = 200$ per test set, seed 20260525.

Test set	Uncalibrated	Calibrated
In-distribution (expect HEALTHY)	1.000	0.875
OOD collapse (expect COLLAPSE)	1.000	1.000
OOD maximum (expect MAXIMUM)	1.000	1.000

The result is specific and bounded. COLLAPSE and MAXIMUM are detected at 100% in both modes: the absolute entropy thresholds alone separate near-one-hot and near-uniform attention from in-distribution attention without any data-derived reference. What calibration adds is the DEVIATION class — the ability to flag in-distribution attention that drifts from its own

nominal band. The cost of that added sensitivity, on this synthetic in-distribution set, is a 12.5% deviation-flag rate (in-distribution accuracy 1.000 → 0.875): the 3σ band flags a minority of well-formed samples as DEVIATION.

We report this trade-off rather than a false-alarm reduction. Calibration is not required for binary collapse/maximum detection, which is reliable from absolute thresholds; calibration is the mechanism that enables the finer DEVIATION class, at a measurable in-distribution flag cost that a deployment would tune via the σ multiplier.

6 Outstanding Measurements

The following measurements are required for a complete evaluation and are documented as missing:

GPU overhead. Protocol P2 (`run_overhead_torch.py`) is provided for community measurement. Expected 3–8× lower than CPU due to bandwidth advantage. Cannot be claimed until measured on actual hardware. We do not claim this number in any result table.

Real training run validation. Protocol P3 (`run_early_warning_torch.py`) trains a small transformer in FP16 without AMP loss scaling. Results on this protocol are pending hardware execution.

Large-model validation. Protocol P4 (`run_gpt2_attention_monitor.py`) runs GPT-2 small (117M parameters). Validation on larger models (GPT-2 medium/large, LLaMA checkpoints) is not reported.

Multi-architecture threshold calibration. The ghost/real thresholds (0.72, 0.15, -0.025, 10^{-4}, -0.3) are derived analytically. Whether these transfer to convolutional, recurrent, or state-space architectures is an open empirical question.

FSDP/DeepSpeed compatibility. The current implementation assumes tracked parameters are accessible on the process running `step_watch()`. Model-parallel and FSDP settings are not tested in v0.4.0.

These gaps are not papered over. Each is matched to a measurement protocol. Results will be reported in subsequent releases as measurements become available.

7 Failure Taxonomy

NRI classifies numerical failures into seven categories based on structural dynamics:

Ghost fault (transient structural deformation). Transient decline in shape and signal scores with restoring force present; self-corrects without intervention. Observable only because NRI maintains a trajectory window. Without trajectory context, a ghost fault is invisible at the magnitude layer.

Real fault (confirmed structural collapse). Shape score below 0.72 with ≥ 2 of three fault signals. Restoring force exhausted; monotonic collapse. The `FaultContext` delivers the last-healthy singular-value snapshot as a structural recovery target.

Oscillatory instability. Alternating GHOST/REAL states without monotonic decline. Not yet a distinct classification state in v0.4.0.

Delayed divergence. Slow collapse over a long window; context quality degrades with staleness; `adjusted_confidence` models the decay.

Silent corruption trajectory. Shape score stabilized below the healthy threshold without triggering a `NaN` event. Invisible to magnitude-based monitors. The `FaultContext` reveals the structural degradation. The most consequential category for long-run generation quality.

Catastrophic propagation. A real fault propagates through the forward or backward pass to downstream tensors. The telemetry event log provides the temporal record for propagation-chain reconstruction.

Attention pathology (inference-time). COLLAPSE ($H \to 0$) or MAXIMUM ($H \to \log n$) in one or more attention heads. Detectable without calibration.

8 Resolution Boundary

The framework terminates at a resolution boundary. When the classification layer confirms a non-finite event, control passes to the Resolution Layer, a proprietary subsystem that returns a defined value in place of the undefined one, allowing the computation to continue.

This layer is maintained in a separate private codebase. Its method and empirical evaluation are deliberately outside the scope of this paper and are the subject of separate forthcoming work; the present contribution is confined to the observability and classification layers evaluated above. No claim, figure, or success rate is asserted for the Resolution Layer in this document.

9 Open Source Strategy and Release Lineage

9.1 Open/proprietary boundary

The open/proprietary split follows the architecture layers:

Open (Apache 2.0): The entirety of Layer A (observability) and Layer B (classification): all mathematical primitives, the three-gate engine, the fault classifier, the inference-time monitor, the telemetry bus, the `FaultContext` protocol, the full benchmark suite, and the NumPy algebraic validation mirror. Approximately 1,100 lines of code across 8 modules. The paper (LaTeX source) is released under CC BY 4.0.

Proprietary: The resolver implementation (Layer C concrete implementation). The `ResolverProtocol` abstract base class documents the interface contract; the implementation details remain private.

9.2 Version lineage

v0.4.0 (current). Core library + 6 benchmarks + this preprint. Benchmark seed `20260525` locked. CPU overhead measurements. No `pypi` package yet.

v0.5.0 (planned). `pip install nonans`. GPU overhead measurements from community protocols. JSON benchmark output alongside PNG figures. CHANGELOG.

v0.6.0 (planned). Extended benchmark coverage. Additional architecture validation (BERT, T5, or similar). Improved calibration API. FSDP/DeepSpeed compatibility testing.

v1.0.0 (milestone). Stabilized API contract. Comprehensive documentation. Citation-ready release. All six benchmark figures reproduced on GPU hardware.

9.3 Repository structure

`nonans/` core library (Apache 2.0), 8 modules; `benchmarks/` 6 benchmarks, NumPy validation suite; `protocols/` 4 PyTorch reproducibility scripts; `figures/` benchmark outputs (PNG + JSON in v0.5.0); `paper/` this preprint (LaTeX + PDF).

10 Licensing Philosophy and Commercial Pathways

The NRI framework is designed as a two-layer commercial stack. The open monitoring and classification layer (Apache 2.0) establishes research credibility, community trust, and ecosystem

adoption. The proprietary stabilization layer (the resolver) is the commercial differentiator: context-informed stabilization using the `FaultContext` record is the capability that transforms structural observability into training continuity.

Dual-licensing model. The open layer is Apache License 2.0: a permissive license with no constraints on commercial use and an explicit patent grant, chosen over a bare-permissive alternative precisely because the patent grant gives downstream commercial integrators clearer terms. Enterprise deployments that require the resolver layer, SLA-backed support, or custom integration operate under a separate commercial license.

Enterprise applicability. NRI is an infrastructure component, not an application. The single-line integration (`model = nonans.wrap(model, optimizer)`) and non-intrusive design (monitoring errors never crash training) make it suitable for integration into existing training infrastructure without architectural changes. The `ResolverProtocol` abstract base class enables custom enterprise resolver implementations without modifying the open layer.

Attribution: what is required, and what is requested. The distinction is deliberate and follows established open-source research practice. The Apache 2.0 license governing the open layer requires, of anyone who redistributes the code: retention of copyright, patent, trademark, and attribution notices; that modified files be marked as changed; and that recipients receive a copy of the license. It does not require citation, does not require preservation of terminology, and imposes no obligation on those who merely read this paper or reimplement the method from its description.

Separately, and as a matter of academic norm rather than license, we *request* that work building on this contribution cite the paper and the `nonans` repository. Compliance with the Apache 2.0 license does not exempt downstream academic or commercial work from standard citation practice when this software or its results are used. The named terminology—NRI, signal score, ghost fault, real fault, `FaultContext`, stabilization budget, structural precursor advantage—is provided as a citation anchor: its adoption in derivative work is the practical mechanism by which attribution propagates, but it is requested practice, not an enforceable requirement. The protectable assets of this work are the copyright in the specific code (Apache 2.0, with its patent grant) and any registered trademark in the project name; the published concepts and architecture are contributed openly.

11 Provenance and Attribution

11.1 Technical vocabulary as attribution anchor

This section documents the named technical contributions of this work as an attribution anchor for future citations, reimplementations, and derivative architectures.

Named framework: Numerical Runtime Intelligence (NRI).

Named architectural layers: Layer A (Numerical Observability), Layer B (Propagation Classification), Layer C (Stabilization Orchestration).

Named components: Gate 1 (norm proxy), Gate 2 (structural scan), Gate 3 (context query), `FaultContext`, `ResolverProtocol`, `Registry`, `TelemetryBus`, `RuntimeMonitor`, `attention_entropy_batched`.

Named fault taxonomy: ghost fault, real fault, oscillatory instability, delayed divergence, silent corruption trajectory, catastrophic propagation, attention pathology.

Named metrics: signal score, shape score, collapse rate, restoring force, momentum alignment, structural precursor advantage, stabilization budget.

Named causal stages: structural deformation onset, restoring force depletion, magnitude manifestation, catastrophic event.

Each of these terms has a precise technical definition in this paper. Their adoption in derivative work implies reference to this framework.

11.2 Version-anchored benchmark results

The benchmark results in this paper are anchored to `nonans` v0.4.0, seed `20260525`. Future versions that alter benchmark results will report results under their own version number and reference this paper as the v0.4.0 baseline. The seed `20260525` is fixed and will not be changed across versions; backward compatibility with prior results is maintained through explicit version labeling.

11.3 Fabrication note

An earlier commercial whitepaper associated with this work cited two arXiv identifiers (2602.23058, 2602.21467) that do not correspond to real papers. These identifiers have been removed from all academic documents. All references in this paper have been verified to exist. Future documents associated with this work should be checked for fabricated citations before publication.

12 Conclusion

We have introduced Numerical Runtime Intelligence (NRI), defined its position in the GPU systems stack, and demonstrated that Shannon entropy of already-computed distributions is a principled, low-cost, unified structural health signal for transformer training and inference.

The framework delivers three empirically bounded results: the training-time and inference-time structural signals are mathematically equivalent (3.18×10^{-7} maximum error); the signal score achieves a 29.6-step structural precursor advantage over gradient-norm monitoring at equal false-alarm rate; and CPU overhead is 15.39% mean on already-resident tensors, declining with model size.

The open library (`nonans` v0.4.0) provides six benchmarks, four reproducibility protocols, and a complete NumPy algebraic validation suite. Outstanding measurements—GPU overhead, real-training lead times, large-model validation—are clearly scoped to future protocol executions.

NRI fills a gap in the GPU systems stack that has existed since the first transformer training run: the gap between what the training framework produces and what a resolver can act on, filled with structure rather than silence.

A Reproducibility Protocols

P1 `run_unified_identity_torch.py`**.** Verifies Proposition 1 via PyTorch. Expected: max error $\leq 5 \times 10^{-7}$ over 10K samples.

P2 `run_overhead_torch.py`**.** Measures entropy overhead vs. attention forward pass on your hardware. Configurable device (`cpu`/`cuda`), n_iter. Reports per-configuration and summary overhead percentages. Results should be submitted in the JSON format defined in v0.5.0.

P3 `run_early_warning_torch.py`**.** Trains a small transformer in FP16 (no AMP loss scaling, high LR) until `NaN`. Reports T_{alarm}, T_{NaN}, and lead time. Tunable via `--lr` and `--alarm_threshold`.

P4 `run_gpt2_attention_monitor.py`**.** Loads GPT-2 small (117M), runs a calibration pass on a user-supplied corpus, monitors attention entropy layer-by-layer. Supports optional collapse injection for detection verification. Requires: `transformers` ≥ 4.30.

B Library API Reference

```
import nonans
```

```
# Training-time: single integration line
model = nonans.wrap(model, optimizer)

# Inference-time monitoring
monitor = nonans.RuntimeMonitor(sequence_length=512)
# calibration pass...
monitor.finalize_calibration()
state, conf, H = monitor.classify_head(
    attn_weights, layer=0, head=0)

# Telemetry
nonans.bus.subscribe(
    nonans.InMemorySink(capacity=1024))

# Custom resolver
class MyResolver(nonans.ResolverProtocol):
    def can_resolve(self, tensor, ctx): ...
    def resolve(self, tensor, ctx,
                budget_ms=5.0): ...
```

C Recommended Citation

Plain text (author-year):

> Makhebi Ahlem (2026). Numerical Runtime Intelligence (NRI): A Framework for Real-Time Observation, Classification, and Selective Stabilization of Numerical Instability Propagation in GPU Transformer Runtimes. *Preprint*, version 1.0.0. DOI: `10.5281/zenodo.20573423`. Repository: https://github.com/makheahlem/nonans.

BibTeX:

```
@article{makhebi2026nri,
  title   = {Numerical Runtime Intelligence ({NRI}):
             A Framework for Real-Time Observation,
             Classification, and Selective Stabilization
             of Numerical Instability Propagation in
             {GPU} Transformer Runtimes},
  author  = {Makhebi, Ahlem},
  journal = {Preprint},
  year    = {2026},
  version = {1.0.0},
  doi     = {10.5281/zenodo.20573423},
  url     = {https://github.com/makheahlem/nonans}
}
```

Canonical citation wording for derivative work:

> "The Numerical Runtime Intelligence (NRI) framework and its `nonans` reference implementation were introduced by Makhebi Ahlem (2026). The signal score, ghost/real fault taxonomy, FaultContext interface, and three-gate monitoring architecture originated in that work."

References

Arora, S., Cohen, N., Hu, W., and Luo, Y. (2019). Implicit regularization in deep matrix factorization. *Advances in Neural Information Processing Systems (NeurIPS)*, 32:7413–7424.

Dong, Y., Cordonnier, J.-B., and Loukas, A. (2021). Attention is not all you need: pure attention loses rank exponentially with depth. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, PMLR 139:2793–2803.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA. ISBN 9780262035613.

Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288. DOI: 10.1137/090771806.

Jain, S. and Wallace, B. C. (2019). Attention is not explanation. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 3543–3556. DOI: 10.18653/v1/N19-1357.

Kloberdanz, E., Kloberdanz, K. G., and Le, W. (2022). DeepStability: a study of unstable numerical methods and their solutions in deep learning. *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 586–597. DOI: 10.1145/3510003.3510095.

Liu, J., Zhang, X., Xu, L., Fang, C., Gu, M., Luo, W., Chai, D., Wang, J., Zhao, Z., and Chen, Z. (2025). Automated detection and repair of floating-point precision problems in convolutional neural network operators. *ACM Transactions on Software Engineering and Methodology*, 34(7), Article 203, 203:1–203:32. DOI: 10.1145/3715104.

Manakul, P., Liusie, A., and Gales, M. J. F. (2023). SelfCheckGPT: zero-resource black-box hallucination detection for generative large language models. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 9004–9017. DOI: 10.18653/v1/2023.emnlp-main.557.

Martin, C. H. and Mahoney, M. W. (2021). Implicit self-regularization in deep neural networks: evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73.

Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, B., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., and Wu, H. (2018). Mixed precision training. *International Conference on Learning Representations (ICLR)*. URL: https://openreview.net/forum?id=r1vgEWe0_.

Miyato, T., Kataoka, T., Koyama, M., and Yoshida, Y. (2018). Spectral normalization for generative adversarial networks. *International Conference on Learning Representations (ICLR)*.

Rodriguez, E., Alvarez-Ramirez, J., and Espinosa-Paredes, G. (2022). A singular value decomposition entropy approach to instability analysis in BWRs. *Nuclear Engineering and Design*, 386:111559. DOI: 10.1016/j.nucengdes.2021.111559.

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30:5998–6008.

Wiegreffe, S. and Pinter, Y. (2019). Attention is not not explanation. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 11–20. DOI: 10.18653/v1/D19-1002.

Yang, X. and Dong, J. S. (2024). Neural Surveillance: live-update visualization of latent training dynamics. *arXiv preprint arXiv:2405.15135*.

Yunis, D., Patel, K. K., Wheeler, S., Savarese, P., Vardi, G., Livescu, K., Maire, M., and Walter, M. R. (2024). Approaching deep learning through the spectral dynamics of weights. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:42011–42035.

Zhai, S., Likhomanenko, T., Littwin, E., Busbridge, D., Ramapuram, J., Zhang, Y., Gu, J., and Susskind, J. M. (2023). Stabilizing transformer training by preventing attention entropy collapse. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202:40770–40803.

Zhang, M., Kong, P., Hou, A., Xia, A., Tuo, W., and Lv, Y. (2022). Identification strategy design with the solution of wavelet singular spectral entropy algorithm for the aerodynamic system instability. *Aerospace*, 9(6):320. DOI: 10.3390/aerospace9060320.